

Empirical Validation of Monte Carlo Convergence to the Black-Scholes-Merton Model of Option Pricing

Eric Varvachtian

August 16, 2026

1 Abstract

While a closed-form analytical solution exists for pricing European options, Monte Carlo simulation of the price for a European Option is a worthwhile topic to explore as the simulation can be compared to the known solution, validating its accuracy. In python, we design a Monte Carlo simulation that prices a European Call and Put option, and then explore how the Monte Carlo price estimate converges to the analytical solution, the rate at which the error of the estimate decreases, and how computation time increases with increased time steps per simulated path. We find that empirically as the number of simulated paths per Monte Carlo run increases, the price estimate converges to the analytical solution. We also find that the error of the Monte Carlo price estimate converges to zero at the theoretical $\mathcal{O}(\frac{1}{\sqrt{N}})$ rate. Finally, we find that increasing the number of time steps per simulated path does not provide any benefits in terms of pricing accuracy, while increasing computation time. Thus, we show that a Monte Carlo simulation converges to the price established by an analytical solution, giving us a useful benchmark for applying Monte Carlo methods to pricing options that do not have a closed-form analytical solution.

2 Introduction

2.1 Mechanics and Structures of Options

According to (Hull, 2022), financial derivatives are contracts whose value depends on the value of some underlying asset. A stock option is a type of derivative. A **Call option** gives the contract holder the right, but not the obligation, to purchase a stock on a given date at a given price. A **Put option** gives the contract holder the right, but not the obligation, to sell an underlying stock on a given date at a given price. Thus, a Call and Put option have an asymmetric payoff structure. As the price of a stock rises, the Call option gains value and the Put option loses value, and vice versa.

We focus on the pricing of so-called European options (these have nothing to do with geography). European options allow the contract holder to exercise the option only at maturity. The Black-Scholes-Merton (BSM) equations give a closed-form analytical solution

for the price of a European Put/Call option. For the BSM equations and our simulations of European option prices, these five parameters are used:

Table 1: Option Pricing Parameters

Symbol	Description	Domain
S_0	Current price of the underlying stock	$S_0 > 0$
K	Strike (exercise) price	$K > 0$
T	Time to maturity (years)	$T > 0$
σ	Volatility of the underlying stock	$\sigma > 0$
r	Annualized risk-free rate of interest, compounding continuously	$r \in \mathbb{R}$

2.2 Black-Scholes-Merton Equations

From (Hull, 2022), the Black-Scholes-Merton equation for a European Call Option (C) is

$$C = S_0 N(d_1) - K e^{-rT} N(d_2) \quad (1)$$

and the equation for a European Put Option (P) is

$$P = K e^{-rT} N(-d_2) - S_0 N(-d_1) \quad (2)$$

where

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}} \quad \text{and} \quad d_2 = d_1 - \sigma\sqrt{T}.$$

We won't concern ourselves much with how these equations actually work, since they are quite complicated. Instead, we rely upon them as the "baseline" for our simulations. If the simulated price is equal to the analytical price derived from these formulas, then we are successful.

2.3 Assumptions

The following assumptions are made for pricing options, and are held throughout.

Assumption 1. *Transaction costs do not exist. (Hull, 2022, p. 251)*

Assumption 2. *Net of trading losses, we assume that all profits from trading are taxed at the same rate. (Hull, 2022, p. 251)*

Assumption 3. *Lending and borrowing happen at the risk-free rate of interest. (Hull, 2022, p. 251)*

Assumption 4. *Volatility and the risk-free rate of interest are constant over the option's lifetime. (Hull, 2022, p. 348)*

Another “assumption” you should make is that when left unspecified, all options discussed in this paper are European. While there exist other types of options, such as American options (that allow for early exercise), and Asian options (whose payoff depends on the average of the underlying stock price over time), we will not concern ourselves with them. However, by empirically validating Monte Carlo for a European option, we have a useful benchmark before applying Monte Carlo methods to these other option types that do not have a closed-form analytical solution.

3 Proof of Theoretical Convergence

We will now prove three things that we will reference throughout the rest of the paper. Assume that the number of time steps $M = 1$ for all of our calculations. We will look at what happens for other values of M in [subsection 5.3](#).

Proposition 1. *For the sequence of simulated path counts $N_k \rightarrow \infty$, the sequence $\frac{1}{\sqrt{N_k}}$ converges to zero.*

Proposition 2. *As $N \rightarrow \infty$, the Root Mean Square Error (RMSE) of the estimator P_{MC} converges to zero at a rate of $\frac{1}{\sqrt{N}}$.*

Proposition 3. *The theoretical RMSE relationship results in a slope of -0.5 on a log-log plot.*

The following definitions hold for all of our proofs: Let P_{MC} be an unbiased Monte Carlo estimator of the true option price, defined as the sample mean of $N \in \mathbb{N}$ *independent* simulated paths. For each path $i \in \{1, 2, \dots, N\}$, let the discounted payoff be defined as

$$Y_i = \begin{cases} e^{-rT} \max(S_T^{(i)} - K, 0), & \text{Call} \\ e^{-rT} \max(K - S_T^{(i)}, 0), & \text{Put.} \end{cases}$$

where each path shares an identical variance $\text{Var}(Y_i) = s^2$. By definition P_{MC} is the sample mean of these simulated paths, so $P_{MC} = \frac{1}{N} \sum_{i=1}^N Y_i$. We let P_{BSM} be the true population mean (the Black-Scholes-Merton price). Finally, suppose that we only deal with a single time step, and because $M = 1$, the distribution of each individual path Y_i is determined solely by the option parameters from [Table 1](#). Similarly, from our definition above, the population mean and variance are also independent with respect to N .

We now prove [Proposition 1](#).

Proof. Let $(N_k)_{k=1}^\infty = (1, 2, 5, 10, 20, 50, 100, 200, 500, 1000, \dots)$ be the infinite sequence of the number of simulated paths. Assume that the sequence is increasing without bound. Obviously, $\lim_{k \rightarrow \infty} N_k = \infty$. Now define

$$a_k = \frac{1}{\sqrt{N_k}}.$$

We ultimately need to prove that

$$\lim_{k \rightarrow \infty} a_k = 0.$$

By definition of convergence of a sequence, we must show that a_k converges to zero if for every error threshold $\epsilon > 0$, there exists a $K \in \mathbb{N}$ such that $k \geq K \Rightarrow |a_k - 0| < \epsilon$. By definition, $a_k = \frac{1}{\sqrt{N_k}}$ and because $N_k > 0$, a_k is always positive. We abuse notation slightly for this proof. For this proof, and this proof only, K is the threshold for our sequence, elsewhere K is the strike price as noted in [Table 1](#).

Let ϵ be arbitrary. Because $N_k \rightarrow \infty$, there exists a $K \in \mathbb{N}$ such that

$$N_K > \frac{1}{\epsilon^2}.$$

Since N_k is increasing, for all $k \geq K$,

$$N_k \geq N_K > \frac{1}{\epsilon^2}.$$

Thus,

$$\left| \frac{1}{\sqrt{N_k}} \right| = \frac{1}{\sqrt{N_k}} \leq \frac{1}{\sqrt{N_K}} < \frac{1}{\sqrt{\frac{1}{\epsilon^2}}} = \frac{1}{\frac{1}{\epsilon}} = \epsilon.$$

Therefore, for every positive ϵ , there exists a $K \in \mathbb{N}$ such that for all $k \geq K$,

$$\left| \frac{1}{\sqrt{N_k}} - 0 \right| < \epsilon.$$

Then, by definition of convergence of a sequence,

$$\lim_{k \rightarrow \infty} \frac{1}{\sqrt{N_k}} = 0. \blacksquare$$

Next, we prove [Proposition 2](#).

Proof. By definition, the root mean squared error of the Monte Carlo estimator is

$$\text{RMSE} = \sqrt{\mathbb{E}[(P_{\text{MC}} - P_{\text{BSM}})^2]}.$$

By definition, the expected value of an unbiased estimator equals the population parameter, thus $\mathbb{E}[P_{\text{MC}}] = P_{\text{BSM}}$.

By definition of variance, $\text{Var}(X) = \mathbb{E}[(X - \mu)^2]$ for some random variable X where μ is the population mean. Setting $X = P_{\text{MC}}$ we note $\mu = P_{\text{BSM}}$ since we defined P_{BSM} as the population mean. Thus, $\text{Var}(P_{\text{MC}}) = \mathbb{E}[(P_{\text{MC}} - P_{\text{BSM}})^2]$. Substitution into the expression for RMSE yields:

$$\begin{aligned} \text{RMSE} &= \sqrt{\text{Var}(P_{\text{MC}})} \\ &= \sqrt{\text{Var}\left(\frac{1}{N} \sum_{i=1}^N Y_i\right)} \quad (\text{By definition of } P_{\text{MC}}) \\ &= \sqrt{\frac{1}{N^2} \text{Var}\left(\sum_{i=1}^N Y_i\right)} \quad (\text{By the constant factor rule of variance}). \end{aligned}$$

Because simulated paths Y_i are independent, by definition the variance of their sum equals the sum of their individual variances. Since we suppose each path shares an identical variance $\text{Var}(Y_i) = s^2$, then s^2 summed N times is simply $N \cdot s^2$ so our equation now becomes:

$$\text{RMSE} = \sqrt{\frac{1}{N^2} \cdot N \cdot s^2} = \sqrt{\frac{s^2}{N}} = \frac{s}{\sqrt{N}}.$$

From the previous proof, we have already established that

$$\lim_{N \rightarrow \infty} \frac{1}{\sqrt{N}} = 0.$$

Recall that the standard deviation s of a single path depends on the option parameters (Table 1) and is independent of the value of N . Thus, the standard deviation s of an individual path is a constant scalar with respect to N , since changing the number of simulated paths obviously does not affect the standard deviation of a given path.

It follows that

$$\lim_{N \rightarrow \infty} \text{RMSE} = \lim_{N \rightarrow \infty} \frac{s}{\sqrt{N}} = s \lim_{N \rightarrow \infty} \frac{1}{\sqrt{N}} = s \cdot 0 = 0.$$

Therefore, RMSE converges to zero as $N \rightarrow \infty$. Furthermore, since s is a constant with respect to N , the dependence of the RMSE on N is $N^{-1/2}$. We express this relationship in Big O notation:

$$\text{RMSE} = \mathcal{O}\left(\frac{1}{\sqrt{N}}\right). \blacksquare$$

Finally, we prove Proposition 3.

Proof. From Proof 2, $\text{RMSE} = \frac{s}{\sqrt{N}}$. This is rewritten as

$$\text{RMSE} = s \cdot N^{-1/2}.$$

Taking the logarithm of both sides yields

$$\log(\text{RMSE}) = \log(s \cdot N^{-1/2}).$$

By the product rule of logarithms, $\log_b(xy) = \log_b(x) + \log_b(y)$, thus we have

$$\log(s) + \log(N^{-1/2}).$$

Applying the power rule of logarithms, $\log_b(M^k) = k \cdot \log_b(M)$, yields

$$\log(\text{RMSE}) = -\frac{1}{2} \log(N) + \log(s).$$

This can be expressed in the form $y = mx + b$, where $y = \log(\text{RMSE})$, $x = \log(N)$, and $b = \log(s)$. It follows by definition of the slope-intercept form of a linear equation that the theoretical slope is $m = -\frac{1}{2}$. Because the vertical axis represents $\log(\text{RMSE})$ and the horizontal axis represents $\log(N)$, we know the theoretical slope of $-\frac{1}{2}$ occurs on a log-log plot. $\blacksquare$

4 Computational Methodology

4.1 Simulation Parameters and Implementation

This chapter outlines how the Monte Carlo simulation works. The link to my GitHub with the full code and further detail is available at [section 6](#).

The python simulation utilizes NumPy (Harris et al., 2020) and Matplotlib (Hunter, 2007) to implement a Monte Carlo pricing engine that can be compared against the true BSM price.

The simulation uses the inputs from [Table 1](#) for pricing the option. Additionally, we have the same N values we looked at in [Proposition 1](#), where N is the number of simulated paths per Monte Carlo run and M is the number of time steps per path such that $M \in \{1, 10, 100\}$. Finally, $\Delta t = T/M$ is the duration of the time step. The option parameters ([Table 1](#)) and simulation parameters (N, M) are grouped in immutable parameter objects. Note Δt is calculated within the `monte_carlo()` function.

It is also important to note that in `main()`, we set an explicit seed to ensure reproducibility of the simulations under NumPy 2.5.1.

There are two repetition parameters defined in `main()`. `convergence_repetitions` is the number of times to repeat each Monte Carlo run for a given number of simulated paths. So, if `convergence_repetitions` = 50 and $N = 10$, then we do 50 Monte Carlo runs where each run simulates 10 different paths. This is explained in further detail in [subsection 4.3](#). The other repetition parameter is `complexity_repetitions`. This is simply the number of times we repeat timing running the Monte Carlo simulation.

Finally, we defined theoretical RMSE in the proof of [Proposition 3](#) but for the actual simulation,

$$\text{RMSE} = \sqrt{\frac{1}{R} \sum_{j=1}^R (\hat{P}_j - P_{\text{BSM}})^2}.$$

Where R is the number of convergence repetitions, j is the index for independent Monte Carlo runs, and $\hat{P}_j = P_{\text{MC}}$ = the Monte Carlo pricing estimate from all N paths in run j .

4.2 Analytical Price Calculation and Monte Carlo Estimation

The program first calculates the BSM prices for the given option parameters utilizing the equations from [subsection 2.2](#).

(Hull, 2022, p. 251) gives the equation for Black-Scholes-Merton geometric brownian motion (Equation 21.17):

$$S_T = S_0 \exp\left[\left(r - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}Z\right], \quad Z \sim N(0, 1). \quad (3)$$

where Z is a normal variable and S_T is the terminal stock price of the underlying stock at the moment of expiration. We then average the Put/Call payoffs over all N simulated paths to obtain our Monte Carlo price estimate. For $M = 1$, this program applies this equation directly from S_0 to S_T . For $M > 1$, the same GBM transition is applied in a sequence of M

intervals of length $\Delta t = T/M$, with each transition beginning from the stock price produced by the proceeding time step.

Thus, `monte_carlo()` is an algorithm that does the following:

1. Generate terminal stock prices S_T .

For $M = 1$ we directly generate the terminal stock price using [Equation 3](#). We do this N times, with a different random Z for every simulated path. If $M > 1$, [Equation 3](#) is applied sequentially over M time steps of length Δt , where each step uses the stock price from the proceeding step. This ultimately gets us to the same terminal GBM distribution as $M = 1$.

2. Calculate the option payoffs based off of the final stock prices, giving us N possible future Call payoffs (C_T) and N possible future Put payoffs (P_T) by:

$$C_T = \max(S_T - K, 0)$$

$$P_T = \max(K - S_T, 0).$$

3. Because these payoffs happen in the future, we discount them back to today using the risk-free rate of interest:

$$C_0 = e^{-rT} C_T, \quad P_0 = e^{-rT} P_T.$$

4. We take the mean of N discounted payoffs, and this mean is our *Monte Carlo estimate of the option's current price*.

$$\hat{C}_{\text{MC}} = \frac{1}{N} \sum_{i=1}^N e^{-rT} C_T^{(i)}, \quad \hat{P}_{\text{MC}} = \frac{1}{N} \sum_{i=1}^N e^{-rT} P_T^{(i)}.$$

Perhaps the key thing to note for our simulation is that N is the number of paths we simulate, while M is the number of time steps per simulated path.

4.3 RMSE Convergence

Our first experiment deals with observing if the Monte Carlo pricing estimator's error decreases at the theoretical $\mathcal{O}(\frac{1}{\sqrt{N}})$ rate we predicted in the proof of [Proposition 2](#). For the simulation of convergence, we hold $M = 1$ constant to isolate the effect that changing N has on the rate of convergence to zero.

For each value of N :

1. We perform 50 independent Monte Carlo runs, where each run generates N independent simulated paths.
2. Each run outputs one Monte Carlo price estimate for the Put, and one Monte Carlo price estimate for the Call by averaging the payoffs of N simulated paths.
3. Each Monte Carlo price is compared to the BSM benchmark.

4. We calculate the RMSE of our 50 independent Monte Carlo pricing estimates.
5. Plot RMSE vs. N .

Here is a (hopefully) helpful example to clear up any potential confusion between “Monte Carlo runs” and the number of simulated paths.

Suppose $N = 10,000$. For the first run, we generate 10,000 paths that map to one Monte Carlo pricing estimate. Then, we generate a new, independent, Monte Carlo run using 10,000 paths that map to a new Monte Carlo pricing estimate. Then, we generate a third, independent, Monte Carlo run using 10,000 paths that map to a third Monte Carlo pricing estimate. This is repeated for 50 total runs (because we set `convergence_repetitions = 50` in `main()`). But, we do this for every N value in our tuple, so we would do 50 runs for $N = 1$, 50 runs with $N = 2$, etc.

It’s important to understand *why* we do 50 total Monte Carlo runs. If we just wanted to price the option, setting N to something like 1,000,000 paths and running the Monte Carlo simulation should give us an accurate price. We do 50 Monte Carlo runs for every count of simulated paths so we have 50 data points at each N count to compare to the BSM price. This gives us a more reliable estimate of the typical RMSE at each value of N than what a single Monte Carlo run would provide.

4.4 Monte Carlo Price Convergence to BSM

The second convergence experiment deals with observing how the Monte Carlo pricing estimate, P_{MC} , converges to the BSM price as N increases. We set $M = 1$ and $\Delta t = T$ to isolate the effect changing N has on the simulation. We should see that at lower values of N the simulation would be rather inaccurate, and the corresponding graph would be far away from the true BSM price. As N increases, we should see that the Monte Carlo pricing estimate converges to the BSM price. We graph BSM as a horizontal dashed line (since the analytical option price is constant).

We use the same N values and 50 Monte Carlo runs that we did previously, except now we are calculating the mean Monte Carlo pricing estimate over those repetitions, instead of the RMSE.

While RMSE shows the rate at which the error of estimation decreases, this experiment looks at visualizing how the estimator itself approaches the analytical BSM benchmark.

4.5 Computation Time and the Effect of M

Finally, we look at how the computational time increases as we vary the number of time steps used to construct each path. Recall that M is the number of time steps per path, and that $\Delta t = T/M$. We look at $M \in \{1, 10, 100\}$. For each combination of M and N values, we run the Monte Carlo simulation twice and then measure the median of the execution times. Since we have N paths, and each path receives M updates, we have roughly $N \times M$ underlying stock price updates in total. We express the computation time in Big-O notation:

$$\mathcal{O}(NM).$$

Now recall that for a European option, the payoff depends only on the terminal stock price S_T . Because the GBM transition is exact, the $M = 1$ simulation already samples S_T from its exact GBM distribution. Increasing M does not provide any additional accuracy. Therefore, increasing M only serves to increase the computational cost.

Since the program isn't run under a controlled environment, just a standard computer, the absolute execution times are obviously hardware dependent. The primary aim of the experiment is to visually examine the scaling relationship with N and M .

5 Empirical Results and Discussion

5.1 Empirical Results of RMSE vs. N

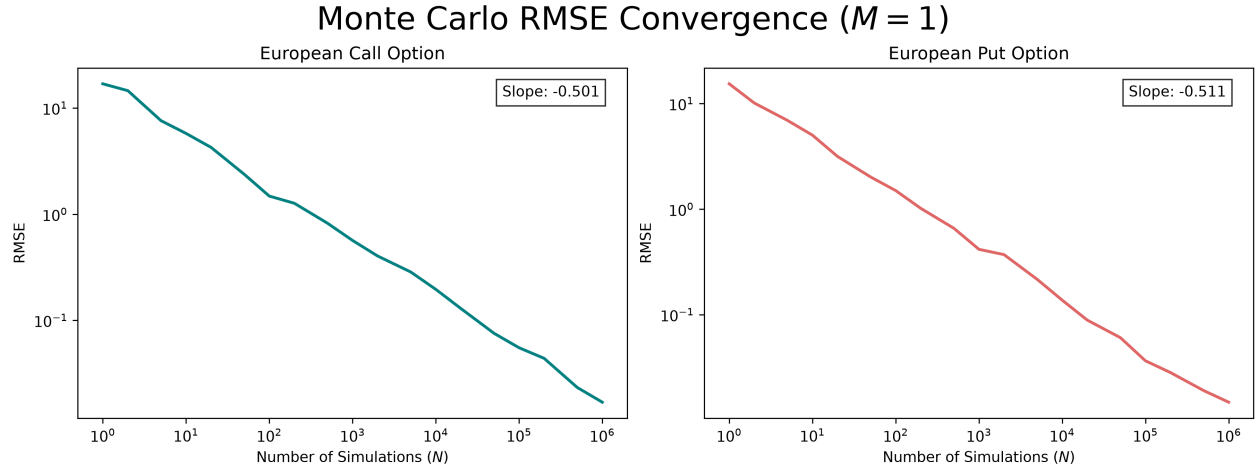


Figure 1: RMSE vs. Number of Simulated Paths (N).

Figure 1 shows how the RMSE converges at the theoretical $\mathcal{O}(\frac{1}{\sqrt{N}})$ rate. Recall from the proof of Proposition 3 that the theoretical RMSE relationship should result in a slope of -0.5 on a log-log plot. Because $m_{\text{call}} = -0.501$ and $m_{\text{put}} = -0.511$, we have $m_{\text{call}}, m_{\text{put}} \approx -0.5$ visually confirming that our simulation achieves the $\mathcal{O}(\frac{1}{\sqrt{N}})$ convergence rate from the theoretical proof.

5.2 Empirical Results of Monte Carlo Price Convergence to BSM

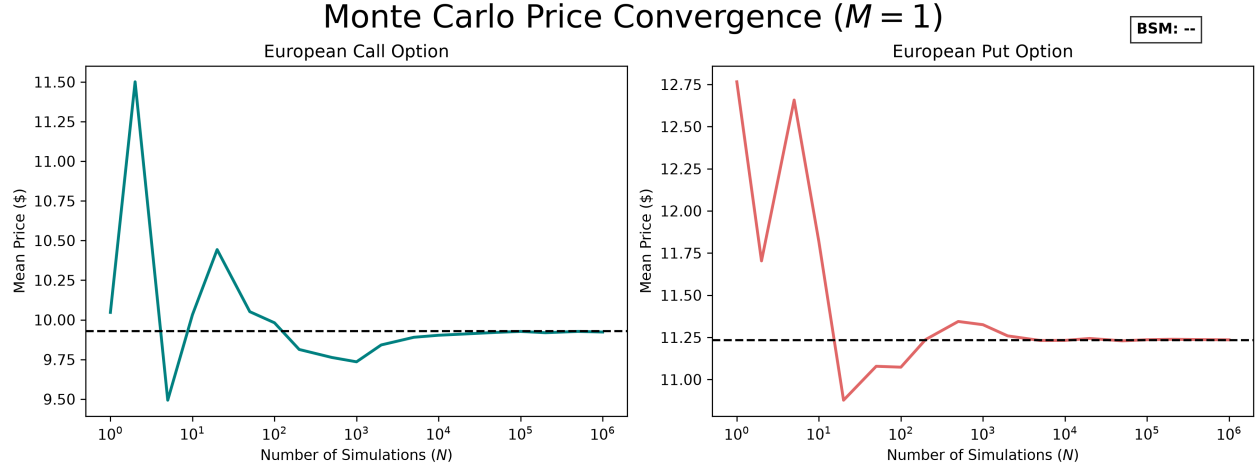


Figure 2: Monte Carlo Price Convergence ($M = 1$).

Figure 2 visually demonstrates how the Monte Carlo pricing estimate converges to the analytical BSM solution as the number of simulated paths increases. The more simulated paths we have, the more accurate our estimate (the mean Monte Carlo price), just like we had expected. Furthermore, while the early swings do look slightly different for the Call and Put option, both eventually settle around the horizontal BSM price as $N \rightarrow 10^6$ paths.

5.3 Empirical Results of Computation Time

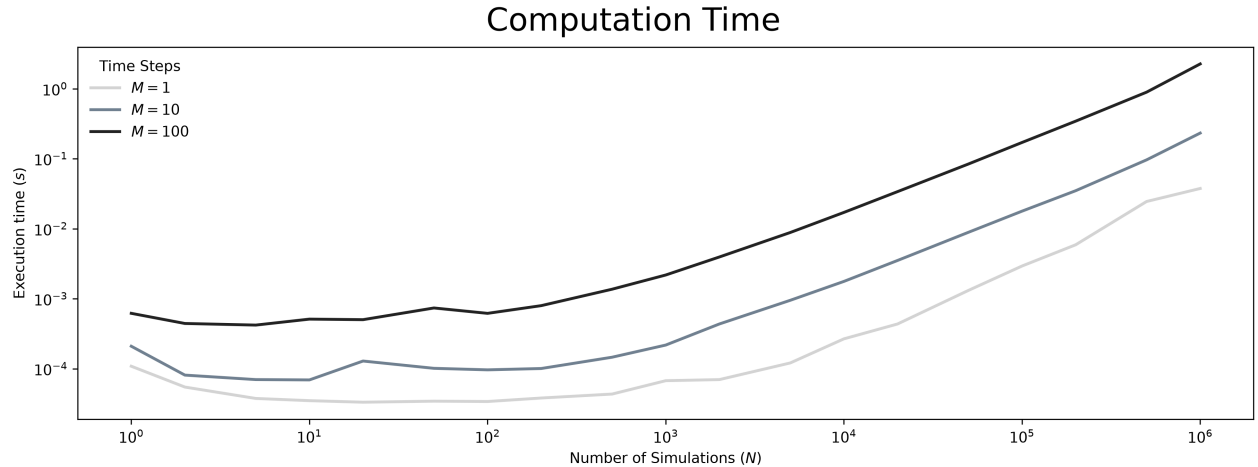


Figure 3: Computation Time

Figure 3 showcases a log-log plot of Execution time vs. Number of Simulations. As we have previously noted, increasing the number of time steps per path does not provide any benefits

in terms of accuracy. The graph empirically validates that increasing the number of time steps per path also increases how expensive the Monte Carlo runs are to compute, since clearly execution time is higher for every larger M value, and this holds true for any amount of paths we simulate.

The results empirically support a $\mathcal{O}(NM)$ complexity for the simulation, as evidenced by the execution time increasing approximately linearly for the parallel upward sloping lines on the log-log plot. Increasing M provides a corresponding increase in execution time, as the line $M = 10$ is around one order of magnitude above $M = 1$, while $M = 100$ is around one order of magnitude above $M = 10$ for larger values of N . This is consistent with the fact that each simulated path requires M GBM transitions, giving approximately NM total calculations for the transitions.

It's important to note that even with averaging runtimes, there is some inherent variance in the computation time. At lower N values, sometimes $M = 1$ may even be slightly more computationally expensive than $M = 10$, however the general trend of execution time being higher for greater M values holds.

6 Conclusion

Together, the experiments demonstrate the expected $-1/\sqrt{N}$ RMSE convergence, convergence of the Monte Carlo estimator to the BSM benchmark, and approximate $\mathcal{O}(NM)$ run-time scaling. In the future, as I continue my studies in mathematics, I hope to revisit some of the stochastic equations we largely ignored, as well as implement Monte Carlo simulations that have an even better RMSE convergence rate, such as Sobol sequences. Finally, I plan on experimenting more with Monte Carlo and seeing what happens when some simulation parameters are relaxed.

Full source code is available at: https://github.com/evavachtian/Mathematics/tree/main/projects/bsm_monte_carlo.

7 References

References

- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hull, J. C. (2022). *Options, futures, and other derivatives* (11th). Pearson.
- Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.